BEST FIRST SEARCH

AIM

To write a Prolog program to implement the Best First Search algorithm for finding a path from a start node to a goal node using heuristic values.

ALGORITHM

1. Start.
2. Define the graph and heuristic values.
3. Initialize the open list with the start node.
4. Select the node with the lowest heuristic value.
5. Check whether it is the goal node.
6. If not, generate its neighboring nodes.
7. Add the neighbors to the open list.
8. Select the node with the lowest heuristic value again.
9. Repeat until the goal is reached.
10. Display the path.
11. Stop.

SOURCE CODE

```
% Best First Search


edge(a, b).

edge(a, c).

edge(b, d).

edge(b, e).

edge(c, f).

edge(d, g).

edge(e, g).

edge(f, g).
```

```
heuristic(a, 7).
heuristic(b, 6).
heuristic(c, 4).
heuristic(d, 3).
heuristic(e, 2).
heuristic(f, 1).
heuristic(g, 0).

best_first(Start, Goal, Path) :-
    search([[Start]], Goal, RevPath),
    reverse(RevPath, Path).

search([[Goal|Rest]|_], Goal, [Goal|Rest]).

search([[Current|Rest]|Queue], Goal, Path) :-
    findall(
        [Next,Current|Rest],
        (edge(Current, Next),
         \+ member(Next, [Current|Rest])),
        NewPaths
    ),
    append(Queue, NewPaths, UpdatedQueue),
    sort_by_heuristic(UpdatedQueue, SortedQueue),
    search(SortedQueue, Goal, Path).

sort_by_heuristic(Paths, SortedPaths) :-
    map_list_to_pairs(path_heuristic, Paths, Pairs),
```

```
    keysort(Pairs, SortedPairs),
    pairs_values(SortedPairs, SortedPaths).

path_heuristic([Node|_], H) :-
    heuristic(Node, H).
```

Query

?- best_first(a, g, Path).

Output

```
?- [best_first_search].
% best_first_search.pl compiled 0.00 sec, 26 clauses

?- best_first(a, g, Path).
Path = [a, c, f, g].
true.

?-
```

RESULT

The Prolog program successfully implements Best First Search and finds a path from the start node to the goal node using heuristic values.